Load-Aware A* Path Planning for Quadruped Robots

Day 14 of 30 · Week 2: Refinement & Integration

NAME : HARINI P (Department of Computer science and Engineering)

1. Nature of Work

Day 14 is the Week 2 checkpoint day — the Methodology section must be complete and presentable to the mentor. Today's work closes out the final two subsections: Implementation Details (covering the environment setup, data structures, and the S_map pre-computation optimisation) and System Architecture (a block diagram showing how all components connect). Together with the Problem Formulation (Day 11), Standard A* (Day 12), and Load-Aware A* (Day 13) subsections, the Methodology is now a complete, integrated draft ready for peer review and mentor feedback.

The implementation details section bridges the gap between the mathematical formulation and the actual code — it explains the design decisions made during implementation that are not captured by the equations alone, such as why S_map is pre-computed, how the grid is represented, and what data structures are used for the open list and closed set. The system architecture diagram provides a single-page visual summary of the entire pipeline.

2. Tasks Completed

- **Complete Methodology draft** — All four subsections written: Problem Formulation, Standard A*, Load-Aware A*, and Implementation Details. Total ~2,200 words, 16 equations, 2 pseudocode listings. Ready for Day 14 mentor review
- **Add implementation details** — Environment setup described (grid class, obstacle placement, neighbour generation). Data structures documented (min-heap, closed set, came_from map, g_score map, S_map). S_map pre-computation rationale and complexity explained.
- **Include environment setup** — Grid environment class documented — dimensions, obstacle probability, seed-based reproducibility, and 4-connected adjacency. Validated that seed=42 produces a solvable grid at 20% obstacle density.
- **Add system architecture** — Block diagram drawn showing the full pipeline: Environment → S_map Builder → A* Search Engine → Path Reconstructor → Results Logger → CSV Output.

3. PROOF OF WORK — METHODOLOGY: IMPLEMENTATION DETAILS

III-E. Implementation Details

This subsection describes the practical implementation of both Standard A* and Load-Aware A* in Python (Google Colab environment). It covers the grid environment class,

the algorithmic data structures, and the key engineering decisions that affect correctness, reproducibility, and computational efficiency.

1) Environment Setup and Grid Representation
The operating environment is implemented as a Python class Environment that encapsulates the grid state and provides the adjacency interface required by both path planning algorithms. The class accepts five parameters at instantiation:
Parameter Type Default Description rows, cols int 10, 10 Grid dimensions. All experiments use 10x10 grids. start tuple (0, 0) Start cell (row, col). Fixed at top-left corner. goal tuple (9, 9) Goal cell (row, col). Fixed at bottom-right corner. obstacle_prob float 0.2 Probability that any non-start/goal cell is an obstacle. Range [0, 1]. seed int 42 Random seed for obstacle placement. Ensures reproducibility across runs.

Parameter	Type	Default Value	Description
`rows`, `cols`	`int`	10, 10	The dimensions of the grid. All experiments are conducted on 10x10 grids.
`start`	`tuple`	(0, 0)	The starting cell, specified as (row, col). It is fixed at the top-left corner.
`goal`	`tuple`	(9, 9)	The goal cell, specified as (row, col). It is fixed at the bottom-right corner.
`obstacle_prob`	`float`	0.2	The probability (ranging from [0, 1]) that any cell, excluding the start and goal cells, will be an obstacle.
`seed`	`int`	42	The random seed used for placing obstacles. This ensures consistent and reproducible results across different runs.

The random generation of obstacles uses Python's random.seed(seed) to guarantee that

seed will generate the same grid (this is necessary for comparing Standard A*
and Load-Aware A*. The start cell and goal cell are always free. The map is represented by a 2D NumPy
integer array with 0 = free, 1 = obstacle.
The grid uses 4-connected (up, down, left, right) neighbours. The get_neighbors(r, c) method only returns free, valid cells, shielding the search algorithms from obstacle and boundary checks
algorithms. This decoupling of environment information in a class, and search algorithms in functions,
makes the code cleaner and easy for Member 2 (Dijkstra/RRT) and Member 3 (testing) to use the same environment class.

2) Algorithm Data Structures

Both A* variants use the same four core data structures throughout the search:
Data Structure Implementation Purpose Open list heapq (Python min-heap) heapq.heappush / heappop Stores (f_score, node) tuples ordered by f(n). O(log n) insert and extract-min. Closed set Python set() closed.add(node) Tracks fully expanded nodes. O(1) membership test. Prevents re-expansion. came_from Python dict() came_from[node] = parent Parent pointer map for path reconstruction. O(1) insert and lookup. g_score Python dict() g[node] = cost Stores best-known actual cost from start to each node. O(1) lookup. S_map (Load-Aware only) Python dict() S_map[(r,c)] = S_value Pre-computed stability penalties for all free cells. O(1) lookup replaces O(k) functi A* Search Algorithm Data Structures

This table summarises the data structures used in the A* search algorithm implementation, detailing their purpose, implementation in Python, and key operations/complexity.

Data Structure	Python Implementation	Purpose	Key Operations / Complexity
Open List	`heapq (Pyth…`	Stores `(f_score, node)` tuples, ordered by the estimated total cost, f(n).	O(\log n) for `heapq.heappush` (insert) and `heappop` (extract-min).
Closed Set	`Python set()`	Tracks nodes that have been fully expanded. Prevents re-expansion.	O(1) for `closed.add(node)` (membership test).
`came_from`	`Python dict…`	A parent pointer map used for reconstructing the optimal path from the goal back to the start.	O(1) for insertion and lookup (`came_from[node] = parent`).

`g_score`	`Python dict…`	Stores the best-known actual cost from the start node to each other node.	O(1) for lookup (`g[node] = cost`).
`S_map` (Load-Aware Only)	`Python dict…`	Stores pre-computed stability penalties (S_value) for all free cells.	O(1) lookup, replacing a more complex $O(k)$ function call.

The S_map (highlighted in purple) is the key optimisation introduced on Day 10. Without it, stability_penalty() is called for every neighbour during every expansion — approximately 300 calls per scenario run, each involving floating-point arithmetic. Pre-computing S_map reduces this to 100 calls total (one per free cell) plus O(1) dictionary lookups during search. This reduced average time overhead from ~1800% to ~60% relative to Standard A*.

3) S_map Pre-computation

The stability map is built once before the search begins. For each free cell (r, c) in the grid, the function computes the foot positions (Eq. 3), the system CoM (Eq. 4), and the stability penalty S(r, c) (Eq. 5), storing the result in a dictionary:

S_map = { (r, c) : S(r, c) for all (r, c) in G | G(r,c) = 0 }

Built in O(rows x cols) = O(100) for a 10x10 grid. Each entry requires O(1) arithmetic (dot product, vector norm). Total pre-computation time: < 0.5 ms in all tested scenarios.

The pre-computation time is negligible relative to the search time, and the memory footprint is O(free cells) <= O(100) floating-point values — trivially small. The S_map is passed into load_aware_astar() as a parameter, making the function pure (no side effects) and testable in isolation.

4) Reproducibility and Seed Validation

All experiments use fixed random seeds to ensure results are exactly reproducible. Both Python's random module and numpy.random are seeded identically to prevent any source of non-determinism. A seed validation scan was implemented (Day 10) to verify that each chosen seed produces a solvable grid at its target obstacle density before running experiments — this prevents the no-path failures encountered with seeds 73, 88, 94, and 101 (Day 10 results).

Scenario Seed Obs% Validated Path Exists 1 42 20% Yes Yes — 18 steps 2 7 35% Yes Yes — 18 steps 3 21 25% Yes Yes — 19 steps 4 33 30% Yes Yes — 21 steps 5 55 20% Yes Yes — 18 steps 6 60 30% Yes Yes — 18 steps 7 73 35% No — blocked Replaced with seed=5 8 88 40% No — blocked Replaced with seed=11 9 94 45% No — blocked Replaced with seed=8 10 101 30% No — blocked Replaced with seed=14

Scenario	Seed	Observations (%)	Validated	Path Exists	Details

1	42	20%	Yes	Yes	18 steps
2	7	35%	Yes	Yes	18 steps
3	21	25%	Yes	Yes	19 steps
4	33	30%	Yes	Yes	21 steps
5	55	20%	Yes	Yes	18 steps
6	60	30%	Yes	Yes	18 steps
7	73	35%	No	No	Blocke…
8	88	40%	No	No	Blocke…
9	94	45%	No	No	Blocke…
10	101	30%	No	No	Blocke…

4. Add system architecture

7. OBSERVATIONS & NOTES

The implementation details section adds real value to the paper. The equations in Sections A-D are mathematically correct, but a reader trying to reproduce the results would be stuck without knowing what data structures were used, how the seed was set, and why S_map was pre-computed. This section fills that gap — it is the difference between a reproducible paper and one that just claims results.

The seed validation table is an honest account of what went wrong. Including the failed seeds (73, 88, 94, 101) and their replacements in the methodology is the right scientific approach — it shows we understand why the failures happened and how we addressed them, rather than silently replacing seeds without explanation.

The system architecture diagram closes the loop visually. After four subsections of equations and pseudocode, a reader can now see the whole pipeline in one figure. The purple colour coding for Load-Aware components immediately communicates which parts are novel versus inherited from Standard A*.